
CODEQUEST 2026

PROJECT PROPOSAL

CampusBook

A Smart Lecture Room Booking System for Universities

Institution	Kwame Nkrumah University of Science and Technology (KNUST)
Department	Computer Science
Course / Event	CodeQuest 2026
Submission Date	31 March 2026
Tech Stack	React Native · Java (Spring Boot) · PostgreSQL
Group Number	102

1. Introduction

Universities are complex institutions with shared physical resources that must be carefully coordinated across departments, faculties, and student bodies. Among these resources, lecture rooms and seminar halls represent one of the most frequently contested assets on campus. At most universities, including KNUST, the process of allocating these spaces remains largely manual: staff send emails, make phone calls, or physically visit administrative offices to reserve a room.

This informal process creates a cascade of problems: double-bookings occur when communication breaks down, rooms sit empty because reservations are not properly tracked, and there is no way for staff to check availability without going through a human gatekeeper. The result is wasted space, frustrated staff, and administrative overhead that could be entirely eliminated through a simple digital system.

CampusBook is a mobile application designed to solve this problem. It provides a centralized platform where staff and students, including course representatives and student leaders who host programs and events, can browse available rooms and submit booking requests from their smartphones, while administrators retain full control over room inventory and approvals.

2. Problem Statement

The current room booking process at university institutions presents several critical pain points:

- No centralized system: room availability is tracked informally through spreadsheets, notice boards, or word of mouth.
- Double-booking conflicts: multiple parties claim the same room for the same time slot with no automatic detection.
- Lack of visibility: staff and students have no way to see in real time which rooms are free without contacting an administrator.
- No formal channel for students: course representatives and student leaders who need rooms for programs and events have no structured way to make requests.
- Administrative burden: administrators spend significant time manually coordinating requests, responding to emails, and resolving clashes.
- No audit trail: there is no reliable record of who booked which room and when, making accountability difficult.

These inefficiencies waste institutional time and resources. A digital solution with automated conflict detection and a clear approval workflow would eliminate most of these problems entirely.

3. Project Objectives

CampusBook aims to achieve the following objectives:

- Develop a mobile application that centralizes all lecture room booking activity across a university campus.
- Implement automated conflict detection to prevent double-bookings at the database level.
- Provide real-time room availability browsing with filtering by capacity, date, time, and facilities.
- Enable a structured approval workflow where administrators can approve or reject requests with documented reasons.
- Maintain a full audit trail of all bookings, cancellations, and status changes for accountability.
- Generate reports on room utilization to help institutions make data-driven decisions about space management.

4. Scope of the Project

4.1 In Scope

-
- User registration and authentication with role-based access (Admin, Staff, and Student).
 - Room management module for administrators to add, edit, and manage room records.
 - Booking request submission, approval, and cancellation workflows.
 - Real-time availability checking with conflict detection logic.
 - Calendar view showing room occupancy across the institution.
 - Basic reporting and analytics dashboard for administrators.
 - In-app notifications for booking status changes.

4.2 Out of Scope

- Integration with existing university ERP or student information systems.
- Payment processing or fee management for room bookings.
- IoT or physical access control integration.

5. Proposed Solution

CampusBook is a mobile application built with React Native on the frontend and a Java Spring Boot REST API on the backend, backed by a PostgreSQL relational database. The system is designed around three user roles (Admin, Staff, and Student), each with a dedicated mobile interface and permission set tailored to their needs.

5.1 Staff Interface

- Browse all rooms with real-time availability status.
- Filter rooms by date, time range, capacity, and available facilities (projector, AC, whiteboard, etc.).
- Submit a booking request specifying purpose, date, and time slot.
- View a personal booking history with current status indicators.
- Cancel upcoming bookings that are no longer needed.

5.2 Student Interface (Course Reps & Student Leaders)

- Self-register on the platform and log in with student credentials.
- Browse available rooms and check availability by date, time, and capacity.
- Submit a room booking request with event name, expected attendance, and purpose.
- Track the status of submitted requests in real time.
- All student requests are routed directly to an administrator for manual review and approval, with no auto-approve.
- Receive in-app notification when a request is approved or rejected, with the admin's reason.

5.3 Admin Interface

-
- Manage the room inventory: add new rooms, edit details, mark rooms under maintenance.
 - Review all pending booking requests and approve or reject with an optional note.
 - View a master calendar of all bookings across all rooms.
 - Access utilization reports: most-booked rooms, peak hours, department usage breakdown.

5.4 Approval Workflow & Conflict Detection

CampusBook enforces a differentiated approval workflow based on user role. Staff booking requests are automatically checked for conflicts and, if none exist, can proceed to approval. Student requests, regardless of conflict status, are always routed to the admin queue for manual review before any room is confirmed. This ensures that student events receive appropriate institutional oversight. The underlying conflict detection engine queries the database for any existing approved booking in the same room on the same date with an overlapping time window, catching both exact and partial overlaps.

5.5 User Experience & Interface Design

Beyond functionality, CampusBook is designed with user experience as a first-class priority. The mobile interface is built to feel intuitive to any university user regardless of technical background.

- Clean mobile UI: a minimal, distraction-free interface built with React Native ensures the app feels native on both Android and iOS, with clear visual hierarchy and consistent design language throughout.
- Easy booking flow: the entire process from browsing available rooms to submitting a booking request is completed in three steps, reducing friction and encouraging adoption across all user types.
- Fast response time: the Spring Boot backend is designed for low-latency API responses, ensuring the app feels snappy even on slower mobile data connections common on campus networks.

6. Business Model

CampusBook is designed not only as an academic project but as a viable, scalable product. The business model is built around a Freemium-to-Subscription strategy, lowering the barrier to adoption for universities while creating a sustainable revenue path as the platform grows.

6.1 Target Customers

- Primary: University administrations and facility management offices seeking to digitize room allocation.
-

-
- Secondary: Polytechnics, colleges, and secondary schools with shared room management challenges.
 - Tertiary: Private training institutes and conference centers with similar booking needs.

6.2 Revenue Strategy

CampusBook adopts a two-pronged revenue approach: a free tier to drive adoption and an institutional subscription tier to generate sustainable income. Individual users, including students and staff, are never charged. Revenue is collected at the institutional level through a single annual contract, making procurement straightforward for university administrations.

6.3 Pricing Tiers

Tier	Price	Key Features
Free	GHS 0 / month	Up to 5 rooms, 20 bookings per month, basic approval workflow, standard support
Campus Pro	GHS 500 / month	Unlimited rooms and bookings, analytics dashboard, full reporting, priority support, custom notifications
Enterprise	Custom pricing	Multi-campus support, API integrations with university systems, dedicated onboarding, SLA guarantee

6.4 Adoption Path

The freemium tier serves as the primary acquisition channel. A university department or student body can onboard CampusBook at zero cost and experience the core value of the platform immediately. Once the institution sees measurable improvement in room coordination, the upgrade conversation to Campus Pro becomes straightforward. This reduces sales friction significantly compared to a direct license-fee model.

6.5 Cost Structure

- Development & maintenance: Covered by the project team during the initial phase.
- Hosting & infrastructure: Cloud server costs (e.g. AWS or DigitalOcean) estimated at GHS 300–500/month at scale.
- Support: community support for Free users, dedicated support for Campus Pro and Enterprise.

6.6 Competitive Advantage

- Built specifically for Ghanaian and African university contexts, not a generic global tool retrofitted for local use.
-

-
- Mobile-first design means zero hardware investment required from institutions, as every student already has a phone.
 - Low entry cost (free tier) removes the biggest objection universities have to adopting new software.
 - Differentiated approval workflows for staff and students reflect real institutional hierarchy and policy.

7. System Design

7.1 Database Schema

The system is built on five core relational tables:

Table	Key Fields	Purpose
users	id, name, email, password_hash, role, department_id	Stores all users; role field holds ADMIN, STAFF, or STUDENT
departments	id, name	University departments for user affiliation
rooms	id, name, building, capacity, facilities, status	Room inventory with facilities and availability status
bookings	id, room_id, user_id, date, start_time, end_time, status	All booking records with status tracking
notifications	id, user_id, message, is_read, created_at	In-app alerts for booking status changes

7.2 API Architecture

The backend exposes a RESTful API organized into four endpoint groups:

Endpoint Group	Base Path	Access
Authentication	/api/auth/**	Public
Room Browsing & Availability	/api/rooms/**	Staff + Student + Admin
Booking Management	/api/bookings/**	Staff + Student + Admin
Admin Controls & Reports	/api/admin/**	Admin only

7.3 Technology Stack

Layer	Technology	Justification
Backend Framework	Java Spring Boot	Robust, enterprise-grade REST API development
Database	PostgreSQL	Relational integrity, strong support for complex queries
Authentication	Spring Security + JWT	Stateless, role-based access control
ORM	Spring Data JPA / Hibernate	Clean entity mapping and query abstraction
Mobile Frontend	Expo (JavaScript)	Cross-platform iOS & Android from a single codebase
HTTP Client	Axios	REST API communication between mobile app and backend
Navigation	Expo Router	Screen routing and stack/tab navigation in the app
Build Tool	Maven	Backend dependency management and project lifecycle

8. Implementation Plan

The project will be developed over four weeks following an incremental delivery model:

Week	Focus Area	Key Deliverables
Week 1	Foundation	Database schema setup, Spring Boot project init, Auth module (register / login / JWT)
Week 2	Core Features	Room management, Booking submission, Conflict detection engine
Week 3	Workflows & UI	Admin approval flow, Calendar view, In-app notifications, Basic frontend
Week 4	Polish & Demo	Reports dashboard, Testing, Bug fixes, Demo preparation

9. Expected Outcomes

Upon completion, CampusBook will deliver the following measurable outcomes:

- Elimination of manual room coordination: all bookings go through a single, centralized system.
 - Zero double-bookings: automated conflict detection makes scheduling clashes technically impossible.
-

-
- Full audit trail: every booking request, approval, and cancellation is logged with timestamp and actor.
 - Improved space utilization: administrators can identify underused rooms and peak demand periods through reports.
 - Reduced administrative workload: staff can self-serve availability checks without contacting an admin.
 - Formal channel for student leaders: course reps and SRC executives can make structured, traceable room requests instead of relying on informal arrangements.

10. Team Roles & Responsibilities

Role	Responsibilities
Backend Developer	Spring Boot API development, database design, conflict detection logic, JWT authentication
Database Administrator	PostgreSQL schema design, query optimization, data integrity constraints
Mobile Developer	React Native app screens, navigation, API integration, booking forms and calendar view
Project Lead / QA	System integration, testing, documentation, demo coordination

12. Conclusion

CampusBook addresses a real, observable problem at university institutions through a technically sound and practically useful solution. The system is scoped to be fully buildable within the project timeline while demonstrating meaningful engineering depth, including a cross-platform mobile frontend in React Native, a RESTful Spring Boot backend, relational database design, role-based access control, and automated conflict detection.

Beyond the CodeQuest 2026 competition, CampusBook represents a deployable mobile product that any university could adopt to modernize its room allocation process. The foundation laid in this project, including the mobile app, REST API, database schema, and conflict detection engine, can be extended in future phases to support push notifications, recurring bookings, and integration with existing university systems.

CampusBook transforms passive room allocation into an intelligent, data-driven system that adapts to user behavior and institutional needs. We are confident that it demonstrates strong problem identification, clear system thinking, and solid technical execution, and we look forward to presenting it.

13. Group Members

The following students are members of Group 102 responsible for the development of CampusBook:

Index No	Reference ID	Name
6128024	21137689	Adjei Desmond
6168224	21138043	Mensah Christopher Lartey
6169924	21138201	Muntari Abubakar Sadiq
6155024	21138217	Dogbey Gideon Kekeli
6150724	21138594	Boateng Adusei Yaw Amanfo